

ORCA Evaluation — Knowledge Transfer Document

Subsystem: ORCA evaluation suite (`evals/`) **Status:** Built and running end-to-end. One command runs everything. Found two real pipeline bugs. **Audience:** Whoever maintains or extends the eval suite next (written assuming you're newish to eval). **Companion docs:** the main `ORCA_KT_Document` (the system being evaluated) and `evals/FINDINGS.md` (results).

Read sections 1-4 to understand *what eval is and what we built*. Sections 5-8 are the file-by-file reference. Sections 9-11 are how to run, what's left, and the issue tracker. Section 12 is the interview/talking-points crib if you ever present this.

0. How to read this document

Section	Covers	Priority
1	What evaluation is, in plain English	Read first
2	Why ORCA needs eval (the silent-failure idea)	Read first
3	The 3-layer architecture	Read first
4	The metrics, defined simply	Important
5	File-by-file walkthrough of <code>evals/</code>	Reference
6	How the judge works (LLM-as-judge)	Important
7	The RAGAS story (why no library)	Important
8	The CI gate (Layer 3)	Reference
9	How to run everything	Do this first hands-on
10	What's done vs what's left	Critical for handover
11	Issue tracker	Critical
12	Talking points / interview crib	Optional

1. What is evaluation? (plain English)

Think of yourself as a teacher checking a smart student (the AI).

A teacher does three things: writes an **answer key**, collects the student's **answers**, and **grades** them against the key. Evaluation is exactly that, automated:

- the "student" = ORCA's RAG retriever and its agents,
- the "answer key" = hand-written test files we created (golden datasets),
- the "grading" = Python scripts that compare and score.

That's all eval is. No magic. The hard part is writing good answer keys and choosing what to measure.

2. Why ORCA needs eval — the silent-failure idea

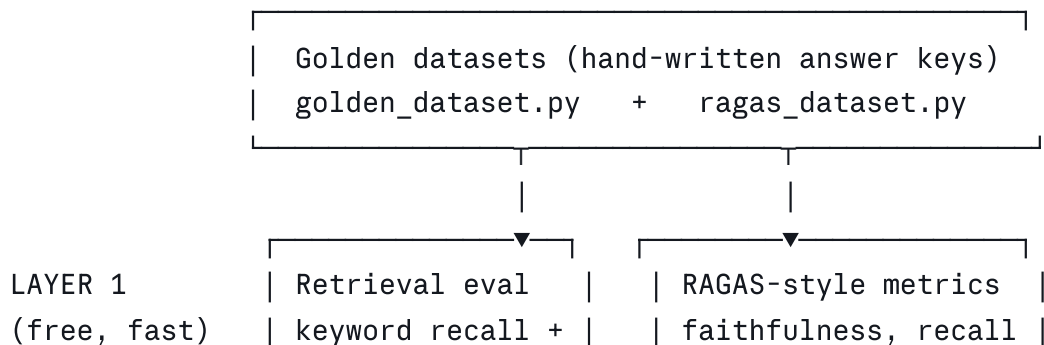
ORCA makes autonomous money decisions (what to reorder, how much, whether a human must approve). Without eval, a prompt tweak or a doc change could silently break a decision and nobody would notice until a wrong order shipped.

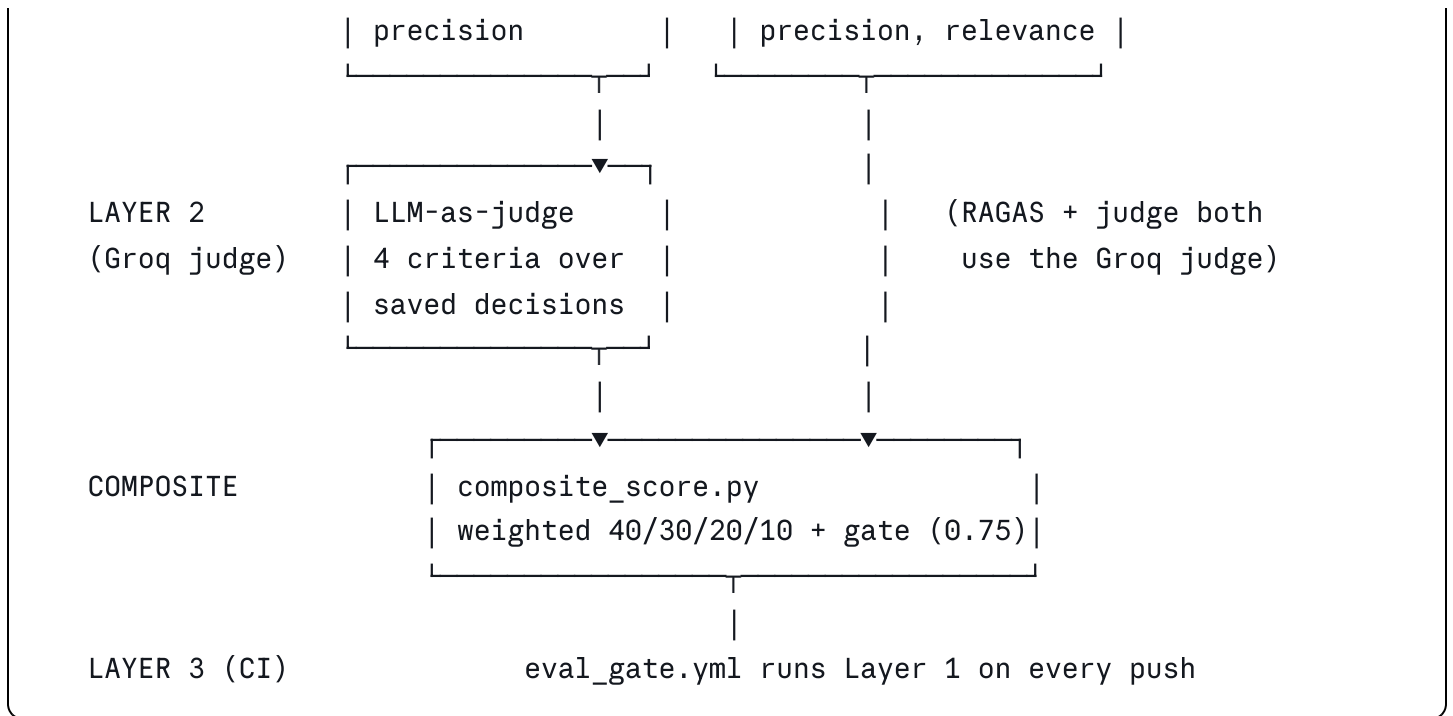
The subtle danger is the **silent failure**. Here's the canonical example every eval engineer knows:

A RAG system can score **high on faithfulness** (the answer matches the retrieved context) while **context recall is low** (the retriever quietly missed documents). The model answers coherently from *partial* context, so faithfulness stays high and everything *looks* fine — but the answer is missing key facts.

The lesson: **you must measure retrieval and generation separately**. If you only check "does the answer match the context," you miss the case where the context itself was incomplete. ORCA's eval measures both — that's the whole reason it has multiple layers.

3. The 3-layer architecture





Each layer goes one level deeper:

- **Layer 1** — are the right *facts present* in what the agent reads? (keyword match, free)
- **RAGAS-style** — is the *answer built from those facts good and grounded*? (LLM-scored)
- **Layer 2** — is the *decision good* across four business rules? (LLM-scored)
- **Composite** — roll it all into one number and a pass/fail gate.
- **Layer 3** — run the cheap layer automatically on every code push (CI).

4. The metrics, defined simply

Layer 1 (retrieval)

- **recall** — of the facts we expect, how many appeared in the retrieved context? (`must_contain` keywords found)
- **precision** — did any *wrong-document* facts leak in? (`must_not_contain` keywords; used sparingly)

RAGAS-style (LLM-scored, 0.0-1.0)

- **faithfulness** (≥ 0.80) — of the claims in the answer, what fraction are supported by the context? (catches hallucination)
- **context_recall** (≥ 0.75) — of the facts in the *ground-truth* answer, what fraction are present in the context? (the silent-failure metric)

- **context_precision** (≥ 0.70) — how relevant is the retrieved context to the question? (signal vs noise)
- **answer_relevance** (≥ 0.75) — does the answer actually address the question?

How these are computed under the hood: break text into individual claims, then ask the LLM yes/no on each claim and take the ratio. Faithfulness decomposes the *answer*; recall decomposes the *ground truth*. Relevance reverse-engineers the question from the answer and compares (needs embeddings). That's the whole trick.

Layer 2 (judge, 1-5 each)

1. **consistency** — do the scored decision, chosen option, cost, and human briefing all agree?
2. **hitl_accuracy** — was human approval triggered exactly when cost exceeded the auto-approve limit?
3. **scoring_accuracy** — did Agent 3 apply the scoring formula correctly?
4. **classa_safety** — was Option B correctly never chosen for a Class A SKU?

Composite

```
composite = 0.40*retrieval_pass + 0.30*context_recall + 0.20*faithfulness +
0.10*answer_relevance
gate passes if composite >= 0.75
```

5. File-by-file walkthrough of `evals/`

<code>evals/</code>	
<code> — peek.py</code>	inspect raw retriever output (what an agent sees)
<code> — peek_db.py</code>	inspect a saved decision field-by-field
<code> — golden_dataset.py</code>	answer key for Layer 1 (12 keyword cases)
<code> — run_retrieval_eval.py</code>	LAYER 1 runner
<code> — ragas_dataset.py</code>	question/ground-truth pairs for RAGAS-style eval
<code> — run_ragas_eval.py</code>	RAGAS-style metrics via Groq
<code> — run_judge_eval.py</code>	LAYER 2 LLM-judge (4 criteria)
<code> — composite_score.py</code>	weighted composite + gate
<code> — eval_main.py</code>	ONE command to run all of the above
<code> — results/</code>	JSON outputs each runner writes (auto-created)
<code> — README.md</code>	quick reference
<code> — FINDINGS.md</code>	results write-up (incl. the bugs found)

Plus `.github/workflows/eval_gate.yml` — the CI workflow (lives in the repo's `.github/`, not in `evals/`).

File	What it does	Needs Groq key?	Importance
<code>peek.py</code>	Print the context string a <code>query_for_agentN</code> returns. Use it BEFORE writing a test case.	No	★ ★ ★
<code>peek_db.py</code>	Print a saved <code>pipeline_log</code> row field-by-field; takes an optional SKU arg. Use to verify judge scores by hand.	No	★ ★ ★
<code>golden_dataset.py</code>	<code>GOLDEN_CASES</code> — list of dicts: each has <code>id</code> , <code>agent</code> , <code>kwargs</code> , <code>must_contain</code> , <code>must_not_contain</code> .	No	★ ★ ★
<code>run_retrieval_eval.py</code>	Calls the real <code>query_for_agentN(**kwargs)</code> , scores recall/precision, writes <code>results/retrieval_latest.json</code> . <code>--ci</code> flag.	No	★ ★ ★
<code>ragas_dataset.py</code>	<code>RAGAS_CASES</code> — list of dicts with <code>question</code> , <code>ground_truth</code> , and optional steering fields (category/abc_class/etc.).	No	★ ★
<code>run_ragas_eval.py</code>	For each case: retrieve context, have the agent model answer, then the judge scores the 4 RAGAS metrics. Writes <code>results/ragas_latest.json</code> .	Yes	★ ★ ★
<code>run_judge_eval.py</code>	Reads saved decisions, judge scores 4 criteria 1-5, writes <code>results/judge_latest.json</code> . <code>--limit</code> , <code>--criterion</code> flags.	Yes	★ ★ ★
<code>composite_score.py</code>	Reads the JSON from the others, applies the 40/30/20/10 weights, prints the gate. <code>--ci</code> flag.	No	★ ★
<code>eval_main.py</code>	Runs steps 1→4 in order as subprocesses. <code>--skip-ragas</code> , <code>--skip-judge</code> , <code>--ragas-limit</code> , <code>--judge-limit</code> , <code>--ci</code> .	(depends)	★ ★

6. How the LLM-as-judge works

The judge is a **stronger** model grading a **weaker** model's work. ORCA agents run on `llama-3.1-8b`; the judge runs on `llama-3.3-70b-versatile`. A judge should be smarter than the student.

`temperature=0` makes the judge **deterministic** — same input gives the same score, so grading is consistent rather than random.

The judge is built directly with `ChatGroq(...)` in the eval scripts, **separate** from `agents/llm_factory.get_llm()`. This is deliberate: we don't change the agents' model (their `.env` stays as-is); the judge just uses a different model name. Swapping the judge (e.g. to GPT-4o for cross-family grading) is a one-line change.

Critical habit: a judge score is *evidence*, not *truth*. LLM judges occasionally misread. Always spot-check a low score by hand with `peek_db.py` before acting on it. (We learned this the hard way — see Issue #2 below: the `classa_safety` criterion was reading the buggy briefing text and false-alarming.)

7. The RAGAS story — why there's no `ragas` library

We *wanted* the real RAGAS library. It would not install in this environment:

- `ragas` hard-imports a Vertex AI class `(langchain_community.chat_models.vertexai.ChatVertexAI)` that doesn't exist in the LangChain version ORCA's agents need.
- A forced reinstall pulled out and replaced langchain, langchain-community, pydantic, click, and others — disturbing the agent stack — and RAGAS *still* failed the same way.
- After that scare we re-verified the pipeline `(from agents.graph import run_pipeline)` imports cleanly). It survived.

Decision: compute the same four metrics natively with the Groq judge `(run_ragas_eval.py)`, no `ragas` package. The metric *definitions* are standard and simple (claim decomposition + LLM yes/no); the library is just convenience. This is the correct senior call — don't take a fragile dependency that risks the production stack when you can implement the method directly.

If someone later wants the real library: it needs a LangChain version compatible with `ragas`, wrapped via `LangchainLLMWrapper` (LLM) + `LangchainEmbeddingsWrapper` (embeddings). You

would still write `ragas_dataset.py` either way — RAGAS never invents test data; you always own the dataset.

8. The CI gate (Layer 3)

`.github/workflows/eval_gate.yml` runs on every push/PR to `main`. GitHub spins up a fresh Ubuntu machine, installs the light deps, and runs `run_retrieval_eval.py --ci`. If the pass rate drops below 70% (or any keyword leaks), the job fails and the email/red-X tells you.

Key design choice: CI uses the **pre-built ChromaDB index** committed at `db/chroma/` — it does NOT re-ingest. Re-ingesting in CI failed because it needs heavy parsers (`docling`, `einops`) that aren't worth installing on a clean runner. Committing the small index (905 KB sqlite + data) is the pragmatic, production-sane choice for a static 5-doc knowledge base.

Only Layer 1 runs in CI today (it's free). RAGAS + judge need a `GROQ_API_KEY` GitHub secret; the steps are present but commented out in the workflow, ready to enable.

9. How to run everything

Prerequisites

1. RAG ingested locally: `python docs/rag/ingest.py --reset`
2. `GROQ_API_KEY` in `.env` (for the LLM-scored layers)
3. For the judge: some pipeline runs saved in `db/orca.db` (click Analyse on a few SKUs in the dashboard first)

One command (the easy way)

```
bash

python evals/eval_main.py
```

Useful variants:

```
bash

python evals/eval_main.py --skip-ragas --skip-judge    # fast: retrieval + composite
python evals/eval_main.py --ragas-limit 3 --judge-limit 5
python evals/eval_main.py --ci                        # exit 1 if composite gate fa
```

Or step by step (when debugging one layer)

```
bash

python evals/run_retrieval_eval.py
python evals/run_ragas_eval.py --limit 3
python evals/run_judge_eval.py --limit 5
python evals/composite_score.py
```

Inspection helpers (use constantly)

```
bash

python evals/peek.py                # what context an agent receives
python evals/peek_db.py SKU00078    # a saved decision, field by field
```

Adding a new retrieval test case (the right workflow)

1. `python evals/peek.py` with the situation you want (edit the call).
2. Read the printed context; pick 2-4 phrases that genuinely prove the right facts are present.
3. Add a case to `GOLDEN_CASES` in `golden_dataset.py` using those *real* phrases.
4. Re-run `run_retrieval_eval.py`; calibrate wording if needed, and leave a one-line comment explaining any change (audit trail).

Golden rule: never write `must_contain` keywords from memory. Always copy them from real `peek.py` output. This is the single most important habit — guessed keywords cause false failures.

10. What's done vs what's left

Done

- Layer 1 retrieval eval — 12 cases, 9 passing (3 need calibration, see below)
- RAGAS-style metrics (Groq-native) — all 4 metrics + thresholds
- Layer 2 LLM-judge — 4 criteria
- Composite score + gate (40/30/20/10, threshold 0.75)
- Layer 3 CI workflow (pre-built index, Layer 1 in CI)
- `eval_main.py` one-command runner
- README + FINDINGS docs

- **Two real bugs found and documented** (see Issue tracker)

Left (all documented, none mysterious)

1. **Calibrate 3 retrieval cases** — `A2-OPTION-RULES`, `A2-CLASSA-NO-OPTIONB`, `A3-ROUTING-RULES`. Wording guesses; fix with `peek.py`. After this, retrieval → ~100% and the composite clears 0.75.
2. **Tune the `classa_safety` judge criterion** — it currently reads the (buggy) briefing text and false-alarms. It should key off the *scored decision* (`capital_decision`), not the briefing.
3. **Online eval not built** — the offline gate is done; grading a sample of live traffic + drift alerting is the next phase.
4. **Enable RAGAS/judge in CI** — add a `GROQ_API_KEY` GitHub secret and uncomment those steps in `eval_gate.yml`.
5. **Judge calibration vs humans** — optionally hand-label a few decisions and confirm the judge agrees, to trust its scores more.

11. Issue tracker

#	Item	Type	Status	Notes
1	Briefing ≠ scored decision (Agent 4)	Pipeline bug (found by eval)	Open, for the pipeline owner	The <code>hitl_briefing</code> names a different option/cost than <code>capital_decision</code> chose. Judge consistency avg 1.8/5; verified by hand on SKU00078 (scored Option A / AED 11,623, briefing said "Option B / AED 6,953") and SKU00033 (scored A / 43,424, briefing said "B / 25,815"). Fix: build the briefing from the winning option in <code>capital_decision</code> .
2	<code>classa_safety</code> false alarm	Eval bug	Open, eval-side	The criterion reads the buggy briefing and flags Class A violations that aren't real (the <i>scored decision</i> was correct). It's caused by Issue #1. Fix: have the criterion read <code>capital_decision</code> 's winning option, and skip SKUs that aren't Class A.
3	3 retrieval cases need calibration	Eval calibration	Open, eval-side	Keyword wording guesses. Fix via <code>peek.py</code> .

#	Item	Type	Status	Notes
4	context_precision low (~0.13)	Measurement nuance	Open, low priority	The retriever returns a large multi-section context, so "fraction relevant" reads low even when recall is perfect. Partly real, partly strict prompt. Worth inspecting <code>RG-CP003-LIMIT</code> (scored all-0). Not alarming.
5	Online/drift eval	Missing feature	Open, future	No live-traffic grading yet.
6	RAGAS/judge not in CI	CI enhancement	Open	Needs <code>GROQ_API_KEY</code> secret; steps are commented in <code>eval_gate.yml</code> .
7	<code>ragas</code> library half-installed in venv	Housekeeping	Open, harmless	A forced install left ragas + shifted deps in the venv. Pipeline re-verified working. If rebuilding the venv, reinstall from <code>requirements.txt</code> . We do NOT import ragas.

12. Talking points (if you ever present this)

- **The two-layer insight:** "I evaluate retrieval and generation separately. A system can score high on faithfulness while context recall is low — the model answers coherently from partial context, so the bug is invisible unless you measure retrieval recall on its own."
- **The bug story:** "My LLM-judge flagged ~7/10 runs for inconsistency. I verified two by hand — the human approval briefing named a different option and cost than the system actually chose. In a real system a human could approve on wrong information. That's the silent failure eval exists to catch."
- **The verify-the-judge story:** "The judge also flagged a Class-A safety violation, but on hand-verification the *scored* decision was correct — the alarm came from a separate briefing bug the judge was reading. One bug created a false reading in a different metric. That's why you verify a judge's score against the raw decision before trusting it."
- **The RAGAS honesty:** "I wanted RAGAS, but it conflicted with our LangChain and a forced install destabilized the agent stack, so I implemented the same four metrics directly with the judge. The definitions are what matter; the library is replaceable — and implementing them myself means I can explain exactly how each is computed."
- **The honest gap:** "The offline gate is done; the missing piece is online eval — sampling live traffic and alerting on drift. That's the next phase."

Final note

The eval suite is built, runs end-to-end with one command, gates in CI, and has already earned its keep by finding real bugs. What remains is calibration and follow-ups — not missing functionality. Keep the golden rule (ground every keyword in real `peek.py` output) and the verify-the-judge habit, and you'll extend this safely.